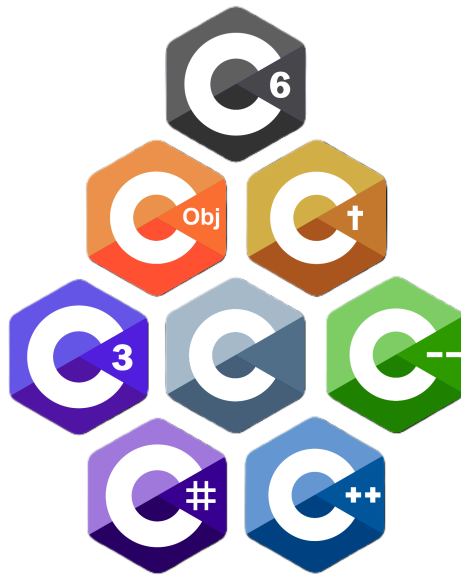




1º CUATRIMESTRE

ACTIVIDAD 3: ENFOQUE MODULAR Y DISEÑO TOP-DOWN - LPR 5º 3º A-B (Presencial/Virtual Asincrónico - Actividades de Contingencia y Continuidad Pedagógica) 2026

OBJETIVOS DE LA ACTIVIDAD

Vamos a aplicar un enfoque Estructurado y **Top-Down** (de lo general a lo particular) en programación.

Aprenderemos a estructurar un programa modular que calcule el área de un círculo utilizando la fórmula matemática:

$$A = \pi \cdot r^2$$

1. ¿Cuál es el enfoque Top-Down?

El diseño **Top-Down** consiste en tomar un problema complejo de ingeniería y dividirlo en partes o módulos más pequeños y fáciles de resolver. En C++, esto lo logramos mediante la creación de **Funciones** diseñadas por el programador. Modularizar nos permite:

- Encontrar errores más rápido (Depuración).
- Reutilizar código (Eficiencia).
- Trabajar en equipo de forma ordenada.

GLOSARIO (Conceptos técnicos que DEBEN dominar)

Como futuros Técnicos en Programación, el vocabulario técnico es su herramienta de presentación profesional:

- **Diseño Top-Down:** Metodología que analiza un problema desde lo general (arriba) hacia lo particular (abajo), subdividiendo el sistema en módulos especializados independientes.
- **Modularizar:** Técnica que consiste en dividir un programa en partes llamadas subprogramas o funciones. Facilita la legibilidad del código, el mantenimiento y la detección rápida de errores.
- **Prototipo de Función:** Es la "declaración de firma" que se escribe antes del bloque main(). Le avisa al compilador de C++ el nombre de la función, qué tipo de dato va a devolver y qué parámetros necesita para ejecutarse.
- **Retorno (Return):** Es el valor o resultado final que la función le devuelve al programa principal (main) una vez que termina de realizar sus cálculos internos.
- **Constante (const):** Variable cuyo valor se bloquea en la memoria RAM y no puede modificarse en ningún momento durante la ejecución del programa (por ejemplo, el valor de π).

ESTRUCTURA DE CARPETAS DEL PROYECTO

Para mantener el estándar de la actividad y el control de versiones con Git, deben organizar la carpeta raíz llamada **CALCULOAREA** exactamente de la siguiente manera:

```
/calculoarea/  
├── .gitignore          <-- Archivo para que Git ignore carpetas de configuración (.vscode)  
├── LICENSE             <-- Archivo de licencia MIT del proyecto (Uso escolar)  
├── README.md           <-- Archivo explicativo del programa desarrollado  
├── docs/               <-- Carpeta exclusiva para documentación  
│   └── InformeProyectoCALCULOAREA.pdf <-- ENTREGABLE (Informe PDF en Normas APA v7)  
├── src/                <-- Carpeta para los códigos fuente del proyecto  
│   └── main.cpp         <-- Código fuente modular en C++  
├── capturas/           <-- Carpeta para guardar las evidencias locales (Imágenes .png)  
│   ├── compilacion_area.png <-- Captura de la terminal compilando y corriendo el programa  
│   └── diagrama_flujo.png  <-- Captura o foto del diagrama de flujo de su programa
```

EL RETO PRÁCTICO: DISEÑO MODULAR EN C++

Crean el archivo main.cpp dentro de la carpeta src/ con el siguiente código base provisto. Deben analizarlo y completar los bloques indicados con comentarios técnicos:

```
/*
=====
Materia: Laboratorio de Programación (LPR)
E.E.S.T. N° 1 "Eduardo Ader" — Vicente López
Curso: 5º Año
Profesor: Prof. York Estudiante: Prof. York
Estudiante: Lopilato Luisana
Archivo: main.cpp
Objetivo: Calcular el area de un circulo mediante funciones modulares
          y simular una diana de tiro en ASCII.
=====
*/

#include <iostream>

using namespace std;

// =====
// 1. PROTOTIPO DE LA FUNCIÓN (Firma de la función)
// Le avisamos al compilador la existencia del módulo de cálculo antes del main
// =====
double calcularAreaCirculo(double radio);

int main() {
    double radioEstudiante;

    // =====
    // REGLA OBLIGATORIA (EVITA EL PLAGIO):
    // Modifiquen la salida de pantalla agregando su Nombre y Apellido reales.
    // =====
    cout << "===== " << endl;
    cout << " CALCULADORA DE AREA MODULAR - ESTUDIANTE: [Su Nombre] " << endl;
    cout << "===== " << endl;

    cout << "> Ingrese el radio del circulo/diana (en cm): ";
    cin >> radioEstudiante;

    // FILTRO DE CONSISTENCIA (Validación del dato geométrico)
    if (radioEstudiante <= 0) {
        cout << "[ERROR] El radio debe ser un valor positivo y mayor a cero." << endl;
        return 1; // Salida controlada del sistema indicando error en consola
    }

    // =====
    // 2. LLAMADA / INVOCACIÓN DE LA FUNCIÓN (COMPLETAR)
    // Convoquen al módulo 'calcularAreaCirculo' pasándole 'radioEstudiante'
    // y guarden el resultado en una nueva variable decimal llamada 'areaFinal'.
    //
    // 💡 SINTAXIS DE REFERENCIA (EJEMPLO GUÍA):
    // Si quisiéramos llamar a una función 'calcularIva' que recibe 'precio'
    // y guardar el resultado en 'ivaTotal', escribiríamos:
    // double ivaTotal = calcularIva(precio);
    // =====

    // [ESCRIBAN SU LÍNEA DE CÓDIGO AQUÍ]
```

```
// =====
// 3. SALIDA DE DATOS (COMPLETAR)
// Muestran en pantalla el área calculada utilizando 'areaFinal'.
//
// 💡 SINTAXIS DE REFERENCIA (EJEMPLO GUÍA):
// Para mostrar texto y variables combinadas en pantalla:
//     cout << "El total es: " << costoTotal << " pesos." << endl;
// =====

// [ESCRIBAN SU LÍNEA DE CÓDIGO AQUÍ]

// =====
// BONUS GAMIFICACIÓN: RENDER DE LA DIANA EN CONSOLA
// Dependiendo del radio que ingresaron, la consola simulará un blanco
// de tiro correspondiente al tamaño calculado.
// =====
cout << "\n[SISTEMA] Dibujando escala de la Diana en la RAM..." << endl;
if (radioEstudiante <= 5.0) {
    cout << "    .---.    " << endl;
    cout << "    / X \ \    -> [DIANA MINI / COMPACTA]" << endl;
    cout << "    \ * /    " << endl;
    cout << "    '---'    " << endl;
} else if (radioEstudiante <= 12.0) {
    cout << "    .---.    " << endl;
    cout << "    / .---. \ \    " << endl;
    cout << "    | / O \ \ |    -> [DIANA ESTÁNDAR DE TIRO]" << endl;
    cout << "    | \ * / |    " << endl;
    cout << "    \ '---' /    " << endl;
    cout << "    '---'    " << endl;
} else {
    cout << "    .---.    " << endl;
    cout << "    / .---. \ \    " << endl;
    cout << "    | / .- \ \ |    " << endl;
    cout << "    || X ||    -> [DIANA GIGANTE DE COBERTURA]" << endl;
    cout << "    | \ '-' / |    " << endl;
    cout << "    \ '---' /    " << endl;
    cout << "    '---'    " << endl;
}

cout << "===== " << endl;
return 0; // Finalización exitosa de Windows (Código 0)
}

// =====
// 4. DEFINICIÓN / DESARROLLO DE LA FUNCIÓN (COMPLETAR)
// Implementen la fórmula matemática de la geometría utilizando variables
// locales y la constante de pi locked en memoria.
//
// 💡 SINTAXIS DE REFERENCIA (EJEMPLO GUÍA):
// Para calcular un valor dentro de la función y retornarlo:
//     double total = base * altura;
//     return total;
// =====
double calcularAreaCirculo(double r) {
    const double PI = 3.1415926535;

    // [DESARROLLEN LA LÓGICA Y RETORNEN EL RESULTADO AQUÍ]

}
```

Comandos de PowerShell para compilar y correr en Windows 10/11:

Abran la terminal integrada de VS Code (Ctrl + Ñ) e ingresen a la raíz de la carpeta **CALCULOAREA**.

1. **Compilen el programa** generando un ejecutable personalizado con el nombre de la aplicación:
`g++ src/main.cpp -o src/area.exe`
2. **Corran la aplicación** generada en Windows:
`.\src\area.exe`

CUESTIONARIO TÉCNICO DE CONTROL

Respondan las siguientes 5 preguntas dentro del informe PDF final que entregarán en Moodle.

Pregunta 1: El Prototipo de la Función

- **Explicación:** En el lenguaje C++, el compilador lee nuestro archivo de código de arriba hacia abajo. Si intentamos usar una función dentro de main() antes de haberla desarrollado por completo, el compilador se perderá y arrojará un error. Por eso, en la línea 15 escribimos un **Prototipo** (una declaración de firma rápida) que le avisa al compilador que la función existe antes de que empiece a leer el bloque principal.
- **Pregunta:** Si borráramos la línea 15 (double calcularAreaCirculo(double radio);), ¿por qué creés que el compilador no sabría qué hacer cuando intente leer la función main?

Pregunta 2: El Filtro de Consistencia

- **Explicación:** En la programación del mundo real, los usuarios cometen errores o ingresan datos inválidos. En nuestro código, usamos una estructura de decisión if (radioEstudiante <= 0) para frenar inmediatamente el programa si el usuario ingresa un radio negativo o igual a cero.
- **Pregunta:** ¿Qué problema físico o geométrico ocurriría si intentáramos calcular el área de un círculo con un radio de 0 cm o de -5 cm? ¿Por qué es importante controlar esto?

Pregunta 3: Proteger Constantes en RAM

- **Explicación:** El número π (Pi) es un valor fijo universal que no debe cambiar jamás. En C++, declaramos const double PI = 3.14159... usando la palabra clave const para que el compilador bloquee ese casillero de memoria. Si por error intentáramos escribir en el código PI = 5.5;, el compilador detendrá la compilación por seguridad.
- **Pregunta:** ¿Por qué considerás que es más seguro para un sistema crítico (como el software de un avión o un sistema médico) declarar valores como "inmutables" (const) en lugar de usar variables comunes?

CRITERIOS DE ENTREGA Y EVALUACIÓN

- **No se aceptan carpetas comprimidas en ZIP o RAR.**
- **Formato de Entrega del informe APA v7 en PDF:** Puede ser grupal (P.I.A) o individual. Si es grupal, el informe APA debe incluir detallado el nombre de cada integrante y qué tarea exacta hizo cada uno. Archivo guardado en su carpeta local como Docs/**InformeProyectoCALCULOAREA.pdf**.
- **Compilación (Local vs. Online):** El uso de VS Code y MSYS2 en la PC es exclusivo para quienes tengan notebook propia con permisos de administrador. (por una limitación de accesos en algunos equipos de laboratorio). Si las PCs de la escuela les bloquea o no les dejan compilar, es obligatorio usar [OnlineGDB](#) para correr el código (hagan las capturas correspondientes).
- **El repositorio:** Sin importar si compilaste en tu computadora o en OnlineGDB, el repositorio propio de GitHub con toda la estructura de carpetas (src/, docs/, etc.) y el código subido con su modificación debe estar creado obligatoriamente. No se aceptan carpetas sueltas.
- **Carpeta de Campo analógica y digital del P.I.A. y [Planilla de Proyectos](#):** Registro obligatorio de todas las actividades relacionadas exclusivamente con el P.I.A., no se deben dejar campos vacíos, se deben justificar con textos coherentes apuntando al P.I.A. (los campos son: **Objetivos para la semana; Entregas Parciales (APA + Código + Capturas); Obstáculos Encontrados y Acciones de Próxima Semana**).

 **Fecha de Entrega:** Prorrogada hasta el **17 de Septiembre de 2026**.

Recursos Útiles (Links Oficiales)

- [Referencia Oficial C++ \(cplusplus.com\)](#)
- [Referencia Oficial C++ \(Microsoft\)](#)
- [Comunidad de Microsoft para Desarrolladores en C++](#)
- [VSCODE](#)
- [Notepad++](#)
- [Diagrama](#) de [Flujo](#) o el [Pseudocódigo](#)